

6-DAY INTENSIVE ENGINEERING PROGRAM

WORKSHOP KICK-OFF

Building Production-Grade Web Applications with React, Laravel, and MySQL. Moving beyond syntax to resilient systems engineering.

Instructor: Sahan madusha

Software engineering & Experience

Founder & Software engineer

Over a decade designing and scaling high-throughput distributed applications, architecting multi-tier cloud systems, and mentoring agile software teams.

- Specialization in distributed micro-services and robust monoliths
- Relational database tuning, query optimization, and high availability
- Translating business requirements into pragmatic tech blueprints

Academic & Technical Core

Formal academic training combined with battle-tested industry practices to bridge theoretical design and production reality.

- Deep foundation in computer science and algorithmic efficiency
- Production-tested engineering methodologies and fault tolerance
- Focus on maintainable code standards and long-term maintainability

SESSION

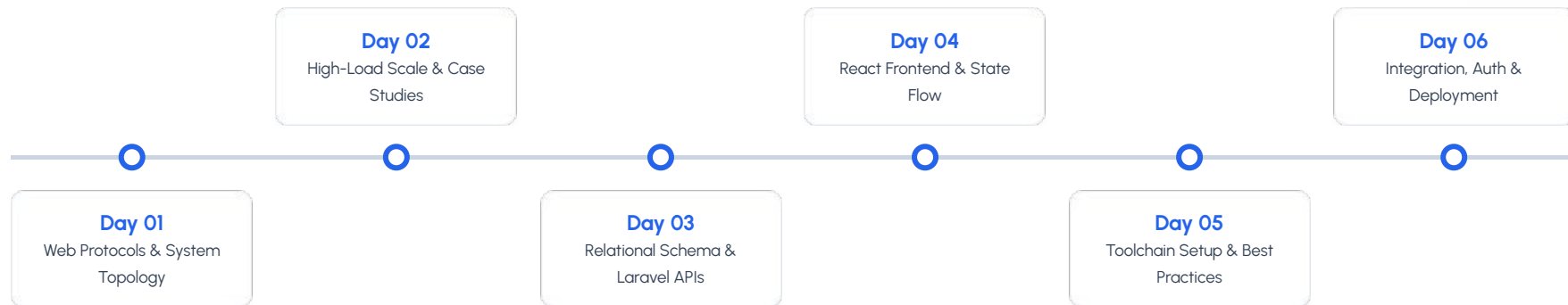
01

Introduction



<https://app.sli.do/event/wCbGTvWNPHbwq3VboKwTFw>

The 6-Day Engineering Roadmap



Engineering Mindset & AI Reality

Errors as Learning Signals

Real engineering involves active implementation, debugging unexpected edge cases, and understanding stack traces rather than passive listening.

- An error code is not a failure; it is concrete diagnostic data
- Debugging builds intuition for architectural edge cases
- System resilience is learned by observing failure modes

Leveraging AI as a Engineer

Code generation is readily accessible. However, blind copy-pasting creates unmaintainable security hazards and technical debt.

- Use AI tools to accelerate prototyping and investigate patterns
- Evaluate generated output with strict architectural scrutiny
- Always understand why a proposed solution functions

Web Development & Core Basics

01

Internet & Web

The global network of networks connecting computers, and the system of information sharing built on top of it.

02

Website vs App

Static pages serving informational content, contrasted with dynamic, interactive applications powering business processes.

03

Browser & URL

The software client visualizing resources, and the structured uniform locator syntax pointing directly to them.

04

HTTP / HTTPS

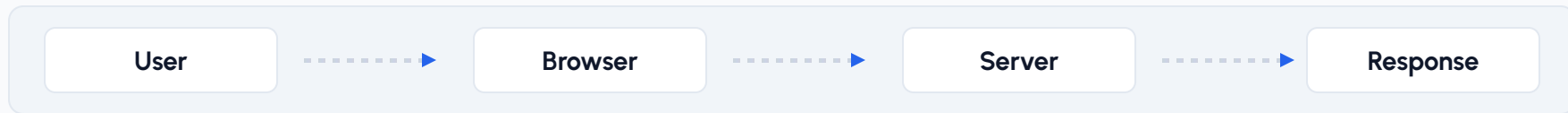
The foundational request-response protocol of the web, secured via cryptographic TLS/SSL layers in HTTPS.

05

Domain & DNS

Human-readable addresses mapped to numeric server IP endpoints through the global decentralized DNS directory.

How the Web Works & Data Flow



01 / ROLES

Client & Server Architecture

The client (typically a browser) initiates communication by requesting resources, while the server listens, processes logic, and serves the requested data.

02 / INTERACTION

Request & Response Lifecycle

Every action is a formal transaction. The client sends structured metadata & payload (Request), and the server delivers status, headers, and body (Response).

03 / PROTOCOL SPECIFICATION

Core HTTP Methods

GET

Requests data from a specified resource without side effects.

POST

Submits data to be processed to a specified resource.

PUT / PATCH

Replaces or partially updates target resource state.

DELETE

Permanently removes the specified target resource.

Key Distinctions: Website vs Web Application

Information Delivery

Website

- Mainly provides information
- Mostly content-focused

Interactive Capabilities

Web Application

- User can perform tasks
 - Supports complex interactions such as login, forms, CRUD, and dynamic data processing
-

Full Stack Integration: How Everything Connects



01 / CLIENT TIER

User & Browser

The interaction cycle starts here. The end user triggers actions inside the browser window, which acts as the local rendering host executing presentation logic and capturing UI input events dynamically.

02 / INTERACTION TIER

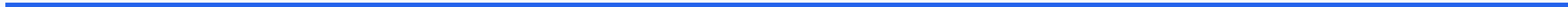
React to API Gateway

The modern React frontend processes the UI state, dispatching highly structured asynchronous HTTP fetch requests towards the API. The API layer securely intercepts, validates, and routes these incoming requests.

03 / APPLICATION TIER

Laravel & MySQL

The robust Laravel backend executes core business rules, evaluates authorization criteria, and performs optimized queries against the MySQL database to retrieve or persist critical application state.



Our Technology Stack: Core Architecture

Layer	Technology	Role & Integration
Frontend	React	01 / CLIENT SIDE Powers the interactive user interface and handles dynamic client-side rendering.
Backend	Laravel / PHP	02 / APPLICATION LOGIC Executes core business services, user auth, and server-side computations.
Database	MySQL	03 / PERSISTENCE TIER Provides transactional, highly structured relational storage for all system state.
API	REST API	04 / COMMUNICATION CONTRACT Exposes robust, secure endpoints for seamless client-to-server data contracts.

Recommendation: Why JavaScript / TypeScript Full-Stack?

Platform	Our Stack	Ecosystem Role
Web	React	Dynamic, responsive component-based UI.
Desktop	Electron + React	Native shell execution using familiar web tech.
Mobile	React Native	Cross-platform iOS and Android native apps.
Backend	Node.js + NestJS	High-concurrency APIs and business logic.
Database	MySQL	Robust, structured relational storage.

01 / SYNERGY

High Code Reuse & One Language

Share logic, types, and validation across clients and backend. Master one ecosystem (JS/TS) to build the entire platform.

02 / CAPABILITY

Multi-Platform & Offline Support

Deploy to Web, Desktop, and Mobile from a unified standard. Enable robust local processing and offline-first client storage.

03 / VELOCITY

Faster Development & Easy Maintenance

Reduce cognitive overhead and context switching. Maintain a single modern code standard for the engineering team.

Key Idea: Learn once. Build everywhere. JavaScript / TypeScript powering Web + Desktop + Mobile + Backend.

SESSION

02

Think Like a Software Engineer

[English](#)[සිංහල](#)[தமிழ்](#)

NATIONAL FUEL PASS

Vehicle

[Vehicle
Registration](#)[Vehicle Login](#)

Non-Vehicle

[Non-Vehicle
Login](#)

If you cannot access your previous profile due to a **mobile number change** or **vehicle ownership transfer**, you may use the '**Vehicle Registration**' option to transfer the profile to your name. Please visit the '**Vehicle Registration**' section and complete the required steps. Once your registration is submitted, you will receive an option to delete the previous profile. Kindly review and validate the details before submitting your request.

If you plan to change your mobile number and have registered using your old number, log in with the old number (OTP) and select "Delete Profile" first. Then re-register using your new mobile number to create a new QR code.

Things to note prior to Registration

- ✓ Enter the Chassis number precisely as per the Certificate of Registration of Motor Vehicles (Case Sensitive)
- ✓ Vehicle registrations with Sri - to only enter the vehicle number in columns Ex: 3 Sri -1234 → **3** **1234**
- ✓ Vehicle category to be selected as per Certificate of Registration of Motor Vehicle. Applicable categories

BIKE	MOTOR CYCLE / LIGHT MOTOR CYCLE
3WHEEL	MOTOR TRICYCLE / MOTOR TRICYCLE VAN / INVALID CARRIAGE
QUADRICYCLE	QUADRICYCLE
CAR	MOTOR CAR / INVALID CARRIAGE
VAN	DUAL PURPOSE VEHICLE
BUS	MOTOR COACH / PRIVATE COACH / OMNI BUS / LIGHT MOTOR COACH
LORRY	MOTOR LORRY / DUAL PURPOSE / LIGHT MOTOR LORRY / M/LORRY(PRIME MOVER) / AMBULANCE / HEARSE / HEAVY MOTOR LORRY
SPECIAL PURPOSE VEHICLE	LAND VEHICLE / SPECIAL PURPOSE VEHICLE / NON AGRICULTURE LAND VEHICLE
LAND VEHICLE	HAND TRACTOR / AGRICULTURE LAND VEHICLE

- ✓ Only one NIC/PP is allowed per vehicle and per mobile no
- ✓ At registration NIC/Name/Address does not require to match Certificate of registration ownership details
- ✓ Company and Gov vehicles/bikes can be registered under BRN category. If require to register multiple vehicles follow below instructions.
 - ✓ Under Nic number create a Unique code for your Institute (Numeric or Alphabetical) and this should be consistent for every vehicle profile.
 - ✓ Any Commercial/Government ministry or Institution/NGO/ Clubs etc to register under BRN category.
 - ✓ Select BRN and follow the registration process.
 - ✓ Multiple vehicles can be registered under a single mobile number.
- ✓ Fuel Liter allocation will be based on vehicle category. Allocation will be announced by Ministry of power & Energy
- ✓ Call 1919 for any registration related assistance

Think Like a Software Engineer: A Simple System Can Hide a Massive Problem

01 / TRAFFIC

Scale & Users

Millions of users concurrently accessing the platform results in massive, unpredictable traffic spikes.

02 / INTEGRATION

External APIs

Third-party dependencies and external services impose strict structural limits on overall capacity.

03 / RESILIENCE

Server Failures

Server-side crashes spark a compounding chain reaction of automated and manual user retries.

04 / LIFECYCLE

Real-world Data

Live production data is highly dynamic, constantly shifting, mutating, and expanding over time.

05 / COMPLEXITY

The Frontend Illusion

A simple, minimalist user interface completely masks the highly complex distributed architecture beneath.

SYSTEM THINKING PARADIGM

Behind the Simple Interface

True software engineering requires designing beyond the static layout. Every interactive system is defined by its architectural boundaries: managing massive execution threads, handling API transaction bottlenecks, designing self-healing retry strategies, and establishing scalable schema designs to cleanly isolate side effects.

Core Rule: Don't just think about features. Think about scale, dependencies, and failure.

Why Did the System Fail? 3 Critical Problems & Failure Chain

01 / BOTTLENECK

Dependency Overload

External APIs couldn't handle massive traffic, creating a massive backpressure bottleneck throughout the entire system architecture.

02 / CASCADE

Retry Storm

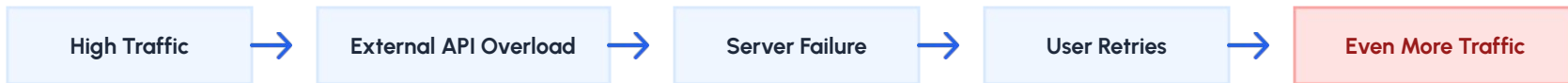
Errors triggered automated retries, while frustrated users repeatedly refreshed the page—generating exponentially more requests and paralyzing the backend.

03 / DRIFT

Wrong Assumptions

Static database schemas, outdated business rules, and stale assumptions failed to adapt to highly dynamic, shifting real-world data patterns.

VISUALIZING THE COMPREHENSIVE FAILURE CHAIN



System Diagnosis: **Interactive systems do not fail in isolation.** Cascade effects must be designed out of the architecture.

Think About the Solution: Engineering ≠ Just Writing Code

01 / ASYNC

Queue & Workers

Decouple heavy tasks using asynchronous processing with robust background queues.

02 / PROTECTION

Rate Limiting

Establish strict API rate limits to protect external dependencies and core services.

03 / INTEGRITY

Smart Cleanup

Isolate data failures and perform target-specific recovery on affected datasets only.

04 / STRATEGY

Phased Rollout

Gradually direct live traffic to verify stability and eliminate unexpected bottlenecks.

05 / DEFENSE

Design for Failure

Incorporate circuit breakers and fallback mechanisms directly into architecture.



Resilience Architecture: Design systems that expect failures. Build self-healing software at scale.

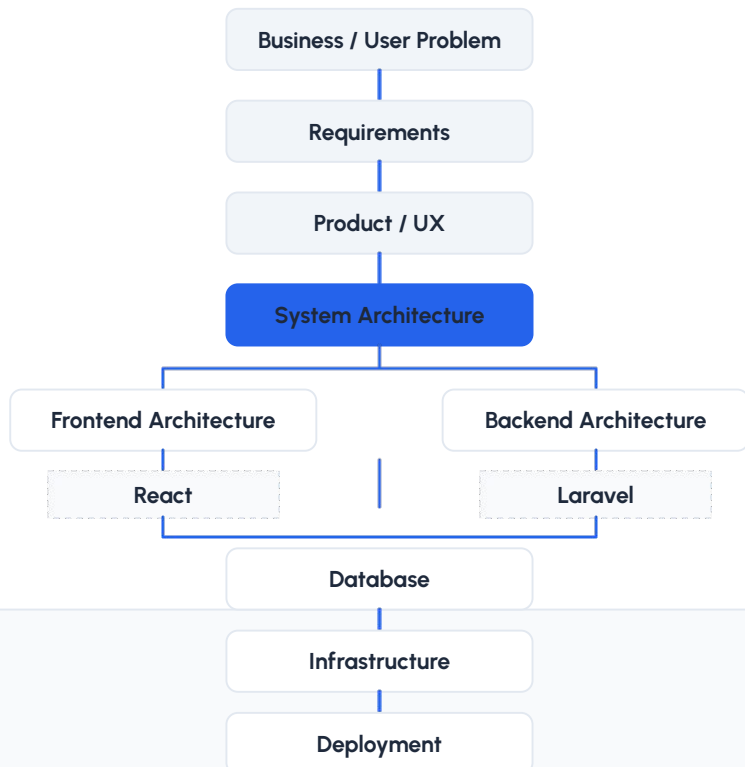
SESSION

03

From Requirements to Software Architecture

From Problem to Software: How Do We Turn a Requirement Into a System?

THE LIFECYCLE OF A REQUIREMENT



THE ARCHITECTURAL PARADIGM

Key Message

A system is defined by more than its code. Developing robust platforms demands clean translation from human problem to structural blueprint.

Good Software $\neq$ Lots of Code

Good Software = Good Structure + Design + Engineering Decisions

What Is Software Architecture? Architecture = The Structure of the System

DEFINING THE BLUEPRINT

Architecture defines:

- Major components: Core building blocks
- Responsibilities: What each part does
- Communication: How components interact
- Boundaries: Strict system separation rules
- Technology decisions: Core stack selection

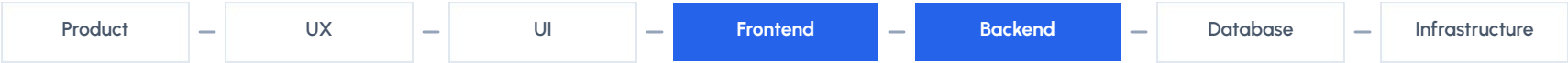
COMPOSITIONAL BOUNDARIES

Design vs. Architecture

	Design	Architecture
Core Question	How does this component work?	How is the whole system organized?
Decision Scope	Feature-level decisions	System-level decisions
Primary Focus	Internal implementation	Major components & boundaries

HIERARCHICAL DEPTH

Architecture Exists at Multiple Levels



System Structure: **Architecture is not a single layer.** It coordinates every decision from user pixel to infrastructure node.

Start With the User: Product Thinking & The UX-to-UI Pipeline

THE DESIGN AND DEVELOPMENT LIFECYCLE

PROBLEM

— USER

— USER NEED

— USER FLOW

— **UX**

— **UI**

— CODE

THE STRATEGIC LAYER

UX: The Journey & Usability

UX represents **the complete journey** of interacting with the product. It governs emotional resonance, efficiency, and clarity of workflow.

- **User Flow:** Strategic pathway mapping out every step
- **Usability:** Natural, intuitive interactions free of friction
- **Feedback:** Clear status validation for user actions
- **Error Handling:** Forgiving paths that prevent failure
- **Accessibility:** Designed to serve all users universally

THE PRESENTATION LAYER

UI: The Interface & Affordances

UI defines **what the user sees and interacts with**. It translates strategic layout blueprints into tactile, functional visual elements.

- **Layout:** Structural composition and information hierarchy
- **Buttons:** Clear visual affordances prompting clean actions
- **Forms:** Inputs structured carefully to capture data cleanly
- **Navigation:** Unambiguous structural pathways of traversal
- **Visual Feedback:** States indicating changes in system focus

Critical Alignment: Good UI can still provide a bad UX. Beautiful interfaces never compensate for a broken journey.

Design for Real Users: Every Screen Has More Than a Success State

THE INTERFACE STATES

Beyond the Happy Path

Interfaces must be robust enough to handle all moments of the user experience. **Don't design only the happy path;** anticipate transitions and system hurdles.

LOADING ↓

SUCCESS ↓

EMPTY ↓

ERROR

ADAPTABILITY

Responsive Breakpoints

Ensure layouts reflow flawlessly across all standard sizes: **Desktop → Tablet → Mobile** without breaking compositional alignment.

ACCESSIBLE DESIGN REQUIREMENTS

Inclusive Architecture

An accessible system ensures that everyone can perceive, understand, navigate, and interact with the interface successfully.

- **Keyboard navigation:** Full structural traversal without mouse
- **Screen readers:** Meaningful semantic markup and ARIA tags
- **Color contrast:** Meeting high legibility guidelines
- **Form labels:** Unambiguous field definitions and context
- **Clear error messages:** Actionable correction feedback
- **Touch-friendly controls:** Generous hit target sizes

A feature is not complete just because the success case works. **Design for resilience.**

Frontend Architecture: React Is a Library — Architecture Is How We Organize It

DIRECTORY STRUCTURE

React Application Workspace

```
src/  
├── Pages/  
├── Components/  
├── Layouts/  
├── State/  
├── Hooks/  
├── Services/  
├── Types/  
└── Utilities/
```

COMPONENT RESPONSIBILITIES

Folder-by-Folder Breakdown

Pages: Major screens and top-level router endpoints

Components: Highly reusable UI building blocks

Layouts: Shared wrappers defining structural page templates

State: Application-wide data stores and state management

Hooks: Custom React functions hosting reusable stateful logic

Services: Modules dedicated entirely to external API communication

Types: Strict TS type interfaces and shared data contracts

Utilities: Pure helper functions and operational helpers

Good structure reduces duplication and makes change easier.

Backend Architecture: **Separate Responsibilities**

DATA FLOW LIFECYCLE

Request Execution Pipeline

```
Client/  
└─ API/  
    └─ Route/  
        └─ Controller/  
            └─ Validation/  
                └─ Service/  
                    └─ Repository/  
                        └─ Database Layer/
```

COMPONENT BOUNDARIES

Layer-by-Layer Separation

LayerResponsibility

Route:Where does the request go?

Controller:Handle request & response

Validation:Is the input valid?

Service:Business rules & logic

Repository:Data access & relationships

Database:Persist the data

Golden Rule: **Controllers should not become the home of business logic.**

Architecture Boundaries & Scalability: **Architecture Is About Boundaries**

SYSTEM LOGIC

Defining Responsibilities

The important question isn't:

"What folder should I create?"

It's:

"Who is responsible for what?"

GROWTH PHASES

As the System Grows

10
Users ↓
10,000
Users ↓
1,000,000
Users

SCALABILITY VECTOR

Architecture may need to support:

- ◆ **Performance:** Efficient resource execution
- ◆ **Scalability:** Handling increased throughput
- ◆ **Caching:** Faster access to frequent data
- ◆ **Queues:** Asynchronous background processing
- ◆ **Rate limiting:** Preventing system abuse
- ◆ **Load balancing:** Uniform traffic distribution
- ◆ **Database optimization:** Reliable data persistence

We don't need to implement all of these today — **but we need to design with them in mind.**

Architecture Boundaries & Scalability: There Is No "Best Architecture"

THE SPECTRUM

Architecture Is About Trade-offs

Simple ←————→ Complex
Low Cost ←————→ High Availability
Fast Dev ←————→ Max Flexibility
Monolith ←————→ Microservices

STRUCTURAL APPROACHES

Monolith vs. Microservices

Monolith Application

Microservices Architecture

Microservices are not automatically better. Choose based on Requirements, Constraints, and Trade-offs.

From Architecture to Team Delivery: **Architecture + Git + Team Practices**

TEAM PRACTICES

A Shared Team Language

A shared architecture gives the team a common language. Everyone understands:

1. Code Location

Where does this code belong?

2. Component Boundaries

What is this component responsible for?

3. Integration & Contracts

How does it communicate with others?

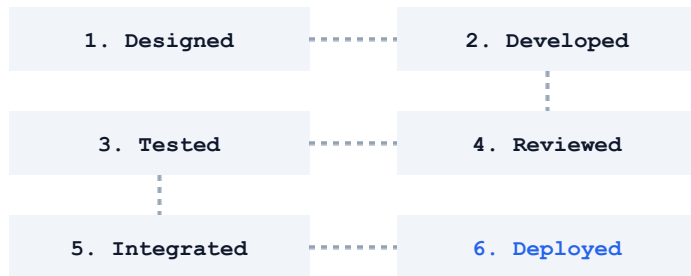
4. Change Management

How do we change it safely?

DEFINITION OF DONE

The Delivery Pipeline

A feature is **NOT** complete because the code was written. It must pass through every stage of delivery:



Think beyond writing code. **Understand the problem → Design the system → Define boundaries → Build with discipline.**

SESSION

04

Engineering Best Practices

Engineering Best Practices: Working Code ≠ Good Code

THE PARADOX

Two Solutions, One Outcome

Two developers can build the exact same feature, and both solutions will compile and function correctly.

But does working code mean the solutions are equally good?

"Code execution is binary, but engineering quality is a spectrum."

THE CRITERIA

What Makes Code Truly "Good"?

- ◆ **Readable** — Easy to follow and comprehend
- ◆ **Maintainable** — Simple to update, extend, or fix
- ◆ **Reliable** — Robust, resilient, and fault-tolerant
- ◆ **Reusable** — Built to avoid duplicating effort
- ◆ **Testable** — Structured to support automated testing
- ◆ **Collaborative** — Instantly clear to anyone on the team

We don't write code only for computers. We write it for people too.

Engineering Best Practices: Write Code for Humans

READABLE & MEANINGFUL CODE

Express Intent Clearly

✗ Hard to understand

```
const x = u.filter(a => a.s === 1);
```

✓ Clear and meaningful

```
const activeUsers = users.filter(user =>  
  user.status === 'active');
```

NAMING CONVENTIONS

✗ Avoid generic names

```
const data; const x; const temp;
```

✓ Use descriptive names

```
const users; const activeUsers; const userProfile;
```

CORE PRINCIPLES

Rules to Code By

◆ Meaningful names

Variables, functions, and classes should tell you exactly why they exist and how they are used.

◆ Clear responsibilities

Each function or module should do one thing, do it well, and have a single reason to change.

◆ Avoid unnecessary complexity

Keep code straightforward. Clean, simple logic is infinitely better than clever, hard-to-read shortcuts.

Clean code isn't just about syntax. **It's about communication, clarity, and discipline.**

SESSION

05

**Development Environment SetupAccount &
Developer Setup**

Session 05 — Development Environment Setup: Account & Developer Setup

ACCOUNTS

Developer Identity & AI Tools

Configure your personal and team access credentials along with core productivity tools prior to writing code:

 **Gmail Account** — Sign in and professional account configuration.

 **GitHub** — Account registration, profile setup, and repository creation.

 **AI Tools Suite** — Set up bookmarks and active sessions for:
ChatGPT · Claude · Gemini · Perplexity AI

DEVELOPMENT ENVIRONMENT

Local Stack Installation

Install and verify the complete runtime engines, databases, tools, and clients required for development:

 **Git**
Version control system

 **MySQL**
Relational database

 **PHP + Laravel**
Backend framework

 **Node.js**
JS runtime & npm

 **Postman**
API testing client

 **HeidiSQL**
Database GUI explorer

Goal: Set up the complete development environment required for the workshop.